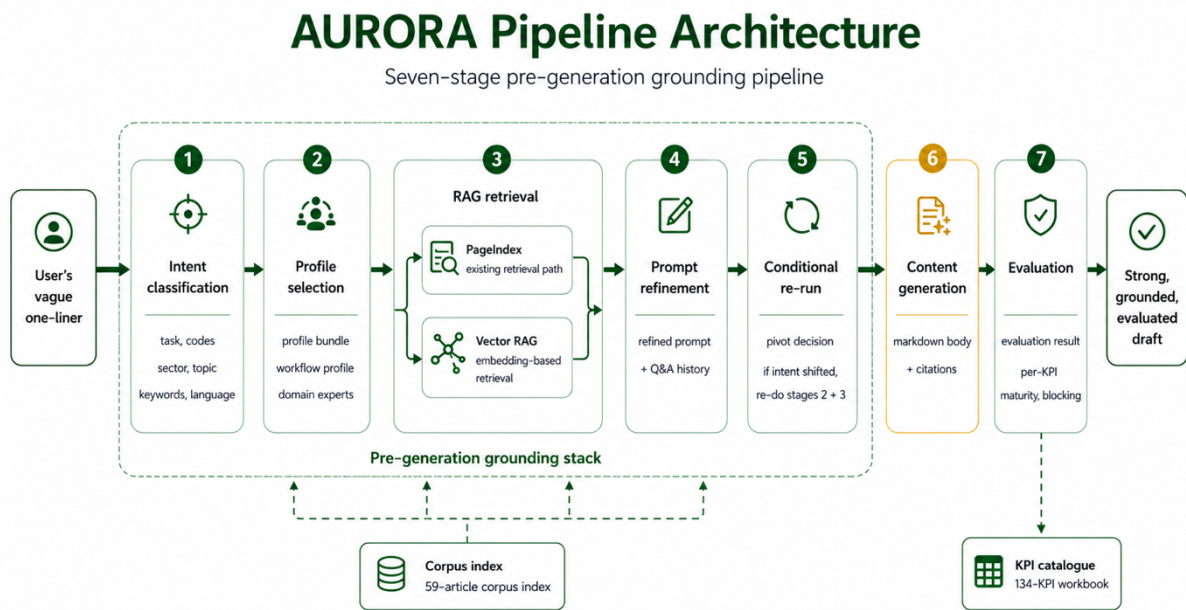


Mid Cycle Report - AI Project

ABN AMRO x AISO

Track 1: Adam Dong, El Yassae, Ilgara Yusifzada, Gaoxiang Ye,
Yuvraj Singh Pathania



PART I: PROBLEM AND ORIGIN	3
1. Summary	3
2. Original Project Statement	3
2.1 What was defined during kickoff	3
2.2 The three pillars	4
2.3 The two key questions	4
2.4 The implied user and scope	4
3. Problem Breakdown and analysis	4
3.1 The chatbot is the benchmark	4
3.2 The real failure: vague in, generic out	4
PART II: SOLUTION AND DECISION PROCESS	5
4. The central solution	5
4.1 The value and three properties	5
5. Decision Process	5
5.1 Insight page as base case	5
5.2 Compliance design	6
5.3 Refinement after RAG, not before	6
PART III: THE SYSTEM	8
6. Pipeline overview	8
6.1 Intent Classification	8
6.2 Profile Selection	8
6.3 RAG Retrieval (Vector Database)	8
6.4 RAG Retrieval (PageIndex)	9
6.5 Prompt Refinement	9
6.6 Content Generation	9
6.7 Evaluation	9
6.8 Inspectability	9
6.9 Auditability	9

Part I: Problem and Origin

1. Summary

- **What AURORA is.** A built, seven-stage pre-generation grounding pipeline followed by a 3-tier evaluator. It turns a vague content request into a prompt that is specific, on-brand, and grounded in approved source material, generates a draft from it, and scores that draft against ABN AMRO's own 134-KPI content workbook.
- **Where we started.** The brief asked for an autonomous agent that could beat ABN AMRO's chatbot at content writing, framed by three pillars (prompt creation, execution, compliance) and two key questions — how to outperform a chatbot, and how to keep the work compliant. It implied a single end-to-end-autonomous agent and a content-team user.
- **The pivot, in one sentence.** The agentic edge is *upstream, not autonomous* (§4): a chatbot fails because the prompt is weak, and autonomy on a weak prompt widens the gap — so the value is in the grounding work done before the model is ever called.
- **The headline decisions:**
 - **Pre-generation grounding over end-to-end autonomy** — invest before generation; generation is then the easy part.
 - **Refinement after retrieval, not before** — clarify against real snippets, turning the loop into discovery rather than blind interrogation .
 - **Compliance split** — designed-in on the input side (profile + corpus boundary + inspectable artefacts) and metric-evaluated on the output side, instead of one end-of-pipeline review.
- **Where we are now: All seven stages are built and wired**

2. Original Project Statement

2.1 What was defined during kickoff

- AURORA (*Autonomous Unified Reasoning & Output Review Agent*), was a POC that together ABN AMRO and AISO wanted to build.
- The goal was simple: **show an agentic/autonomous workflow could beat ABN AMRO's current chatbot at writing tasks.**
- So it was a **demo not a product**, the point was to prove an idea, not to ship one.
- And we had **complete creative freedom** in how to prove it.
- The kickoff set the goal, not the method. That is way the many choices are being tested out and analysis during this demo, not deviation of the final main goal, there was no fixed design to deviate from.

2.2 The three pillars

- The kickoff defined the work as three pillars:
 - **Prompt creation:** the agent writes/improve its own task prompt instead of relying on the user to write a good one.
 - **Execution:** the agent runs the task and produces a draft
 - **Compliance checking:** the agent check its own output against company guideline.

2.3 The two key questions

- The success of this POC was framed by two questions:
 - **How can an autonomous agent beat a chatbot at writing tasks?**
 - **How do we keep retrieval and generation compliant with company guidelines?**

2.4 The implied user and scope

- Two assumptions were never stated during the kickoff, but both matters for the AURORA we were asked to build, before any decision about how to build it:
 - **Implied user - the content team:** the writing task in the kickoff are the ones ABN AMRO's content team does.
 - **Implied scope - full autonomy:** "Autonomous agent" + the three pillars implied one agent owning the whole chain, write the prompt, run it, review the result, with the human mostly out of the loop.
- Both assumptions were reasonable, but neither was required; the open-ended nature of the task allowed either choice. Thats why in the later section we will show our decision process and choice.

3. Problem Breakdown and analysis

3.1 The chatbot is the benchmark

- Content team already have a tool, a conversational chatbot (ChatGPT/Copilot), the kickoff asks us to beat it.
- So before any designing, we have to be precise about **why** it underperforms on content writing.

3.2 The real failure: vague in, generic out

- The chatbot's problem is not that it is slow or lacks autonomy. The problem sits upstream of the model.
- A user types a one-line request, the model has no way to know about: brand voice, writing style, which sources are allows, or what has already been published

- It fills those gaps with the averages of its training data and returns something fluent and generic.
- For content writing, generic is not ideal, the draft has to be sound on brand, fact-checked against approved sources, and re-grounded in real material.

3.3 Adding autonomy makes it worse

- Point an autonomous agent at a vague, ungrounded prompt and it does not steer back toward the brand, it runs further in the wrong directions. With more steps, more output, and a self-review against criteria it also had to guess.
- More autonomy on a bad prompt with less human in the loop, does not close the gap, it widens it and hides it.

3.4 What is the actual problem

- The weak link is not generation, but the deficiency is along three axes:
 - Not specific: the request is a one liner without context, real task is implicit.
 - Not on-brand: nothing has injected the content team's standards
 - Not grounded: nothing ties the request to approved source material
 - So the real question is: ***how do you reliably turn a vague request into a specific, on-brand, grounded prompt before generation runs, in a way ABN AMRO can inspect?***
-

Part II: Solution and Decision Process

4. The central solution

4.1 The value and three properties

- The value of an agentic workflow for content generation is the work done before the model is called, make the prompt strong before generation, not autonomous
- We design the the property such that they are:
 - **Specific:** the implicit task made explicit through clarifying Q&A
 - **On-brand:** the content team's standard injected, not guessed
 - **Grounded:** anchored to approved source material the user has actually seen.

5. Decision Process

5.1 Insight page as base case

- **Question:** Which channel, task, and corpus does the PoC ground on?
- **Options:**

- *Internal corpus*: **richer and closer to real content work, but access is gated by an internal-compliance review that would itself consume the PoC's time.
- Chatbot/Voice-bot: access to chatbot itself and conversational document require compliance review, no clear reward/evaluation metric.
- Web-content Insight page: rich public data with specific domain and topics, very clear user tasks, only limited internal corpus required to improve the final quality.
- **Decision**: Public insight-page articles as the initial corpus, with internal writing reference and draft for content generation, and KPI list of result evaluation.
- **Reason**:
 - The articles are publicly published, so not compliance check.
 - The use case is concrete enough to demo end to end.
 - And nothing in the pipeline is insight-page-specific — intent, profiles, retrieval, and refinement are corpus-agnostic, so the corpus is swappable later.

5.2 Compliance design

- **Question**: The kickoff made compliance an end-of-pipeline self review. Keep it as one final check or split it?
- **Options**:
 - *Single end-of-pipeline review*: the agent review its own output once, at the end
 - *Split compliance*: constrain the input by construction and score the output downstream
- **Decision**. Split: input-side designed-in, output-side evaluated.
- **Reason**:
 - This is how compliance actually works in a bank, you do not only check at the end; you constrain what can go in.
 - Input-side: the active profile injects the content team's standards, the corpus boundary fixes what sources are usable, and every stage emits an inspectable artefact, so the prompt reaching the model is aligned by construction.
 - Output-side: a downstream evaluator scores the draft against ABN AMRO's KPI metrics. Both halves of the original pillar are preserved; only their position changed. A single end-of-pipeline review cannot catch an input that was never constrained.

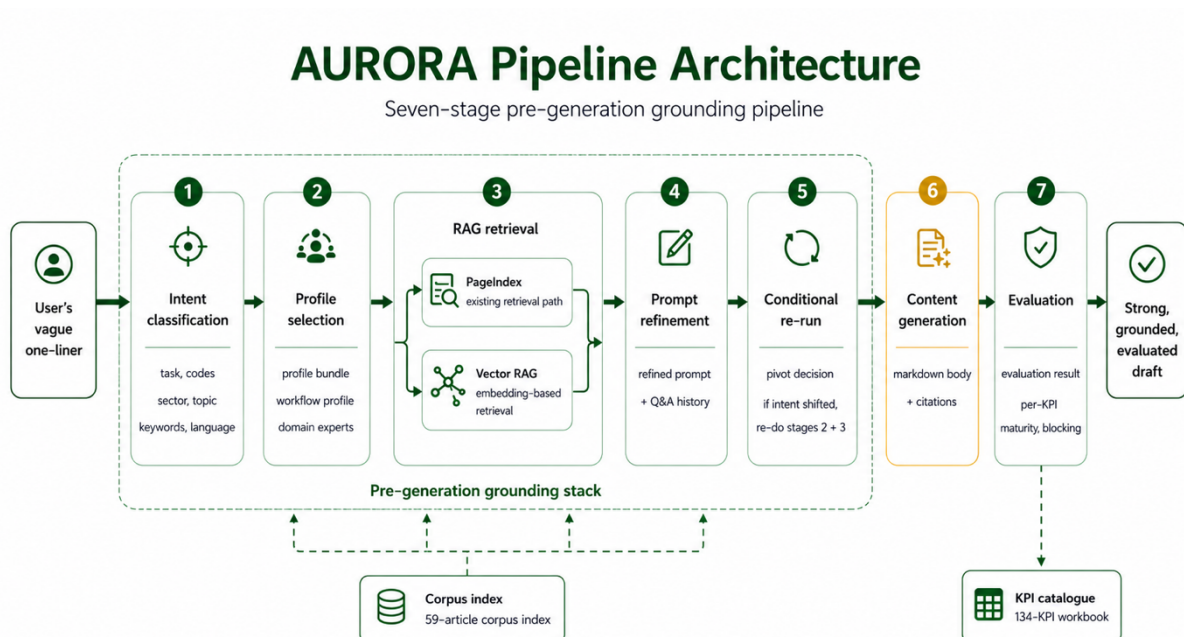
5.3 Refinement after RAG, not before

- **Question**. The clarifying-question loop can run *before* retrieval (clean the question, then look) or *after* it (look, then clarify against what was found).
- **Options considered**.

- *Refine before RAG* — grill the user into a precise prompt, then retrieve once.
 - *Refine after RAG* — retrieve on the raw prompt, show snippets, then clarify against them.
 - **The decision.** Refine after RAG.
 - **Reason.** Refining after retrieval turns the loop from *guessing* into *discovery*:
 - *Grounded beats blind questions.* After retrieval the model can ask about real articles it just found ("we have a piece on EU cyber law — want that angle?"). Before retrieval it can only ask generic questions ("what's your timeframe?") the corpus may not even be able to satisfy.
 - *Profile selection already gives shape.* By refinement time the user has seen which workflow and domain experts fired — that tells them *how* the task will be tackled; snippets tell them *what* is actually available. Both make for sharper questions.
 - *Most users under-specify because they don't know what is possible.* Showing snippets first reveals the surface area and lets the user react ("oh, you have that — I want that angle"). Refinement becomes discovery, not interrogation.
 - **Tradeoffs accepted.**
 - *Wasted compute on pivots* — if the user pivots during refinement, the first retrieval is partly wasted.
 - *One extra latency round* — refinement is a blocking dialog after snippets render. Mitigated by a **Skip refinement** option at every step.
 - *Refinement is anchored to the initial retrieval* — if that retrieval is bad, questions anchor to bad snippets.
-

Part III: The System

6. Pipeline overview



6.1 Intent Classification

- Turn the raw user prompt into a structured result, which tasks are wanted, the sector, topic keywords, and output language. Everything downstream reads structured fields not, free text

6.2 Profile Selection

- Rule-matches the intent to a profile bundle: one workflow profile (drafter / reviewer / curator) for *how to work*, plus zero-or-more domain-expert profiles for *sector voice and authority*. This is where the content team's editorial standards enter the run.

6.3 RAG Retrieval (Vector Database)

- **Purpose:** Pull a handful of prior published articles that are semantically close to the user's task. These become the **style references** the LLM uses to match ABN AMRO's voice and structure during generation. This stage is what makes the prompt *grounded* in approved, real material — directly addressing the third property from §4.1.
- **Source corpus:** 59 ABN AMRO TMT-sector insight articles, English-translated by GPT-5 (the translation step is separate work, upstream of this stage).

6.4 RAG Retrieval (PageIndex)

- PageIndex is a smart, structured table of contents for the bank's approved article library — instead of shredding articles into anonymous fragments the way conventional search does, it keeps the library organised by article and section, the way an editor would navigate it..

6.5 Prompt Refinement

- Rewrites the vague request into a strong, grounded prompt using the profile and retrieved snippets, recording every clarification turn.
- **Conditional re-run:** Checks whether refinement shifted the intent (a *pivot*). If so, re-runs stages 2 and 3 against the new intent so generation isn't grounded on stale context; otherwise a no-op.

6.6 Content Generation

- The deliberately-easy step: produces the markdown draft with citations from the now-strong prompt and grounded context.

6.7 Evaluation

- Scores the draft against the 3-tier KPI catalogue — per-KPI results, maturity by category, and any blocking compliance failures.

6.8 Inspectability

- Every stage emits a typed artefact that carries its own human-readable justification, not just a result. A reviewer never has to ask "why did it do that", the artefact says so.

6.9 Auditability

- Every decision the system makes is recorded and can be reconstructed after the fact. For any piece of content it produces, you can trace exactly what the user asked for, how the system interpreted it, which editorial standards and which approved source material were applied, and what the user clarified along the way.
- If the request changes part-way through, the new steps are added alongside the old ones rather than replacing them, so the full history always survives.